

EXPERIMENT 14

TCL Commands – COMMIT, SAVEPOINT, ROLLBACK

Aim:

To learn how to use various TCL commands Commit, Savepoint and Rollback SQL commands

Procedure and Syntax:

Transaction Control Language (TCL) commands are used to manage transactions in the database. These are used to manage the changes made to the data in a table by DML statements. It also allows statements to be grouped together into logical transactions.

COMMIT:

- COMMIT command is used to permanently save any transaction into the database.
- When we use any DML command like INSERT, UPDATE or DELETE, the changes made by these commands are not permanent, until the current session is closed, the changes made by these commands can be rolled back.

To avoid that, we use the COMMIT command to mark the changes as permanent

ROLLBACK:

- This command restores the database to last committed state.
- It is also used with SAVEPOINT command to jump to a savepoint in an ongoing transaction.
- If we have used the UPDATE command to make some changes into the database, and realize that those changes were not required, then we can use the ROLLBACK command to rollback those changes, if they were not committed using the COMMIT command.

Syntax:

ROLLBACK;

ROLLBACK TO savepoint_name;

SAVEPOINT:

- SAVEPOINT command is used to temporarily save a transaction so that you can rollback to that point whenever required.

Syntax:

SAVEPOINT savepoint_name;

Question 1

Create a table Patient_Transaction and insert patient records. Use COMMIT, SAVEPOINT, and ROLLBACK commands to demonstrate transaction control. Perform an update and insert records between savepoints, then rollback to different savepoints and display the results.

OUTPUT:

1)

```
CREATE TABLE Patient_Transaction
(
    Patient_ID INT PRIMARY KEY,
    Patient_Name VARCHAR(20),
    Age INT,
    Disease VARCHAR(20)
);
```

```
INSERT INTO Patient_Transaction
VALUES
(1, 'Arun', 25, 'Fever');
```

```
COMMIT;
```

```
UPDATE Patient_Transaction
SET Patient_Name = 'Arun Kumar'
WHERE Patient_ID = 1;
```

```
SAVEPOINT A;
```

```
INSERT INTO Patient_Transaction
VALUES
(2, 'Priya', 22, 'Diabetes');
```

```
SAVEPOINT B;
```

```
INSERT INTO Patient_Transaction
VALUES
(3, 'Rahul', 30, 'Asthma');
```

```
SAVEPOINT C;
```

```
SELECT * FROM Patient_Transaction;
```

Result Grid  Filter Rows: Edit: 				
	Patient_ID	Patient_Name	Age	Disease
▶	1	Arun Kumar	25	Fever
	2	Priya	22	Diabetes
	3	Rahul	30	Asthma
*	NULL	NULL	NULL	NULL

```
552 • ROLLBACK TO B;
```

```
553
```

```
554 • SELECT * FROM Patient_Transaction;
```

```
555
```

```
556 • ROLLBACK TO A;
```

```
557
```

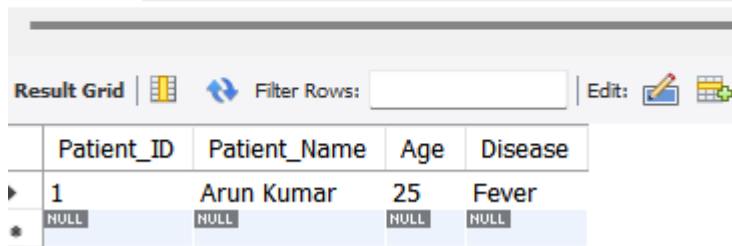
```
558 • SELECT * FROM Patient_Transaction;
```

Result Grid  Filter Rows: Edit:    Ex				
	Patient_ID	Patient_Name	Age	Disease
▶	1	Arun Kumar	25	Fever
	2	Priya	22	Diabetes
*	NULL	NULL	NULL	NULL

556 • **ROLLBACK TO A;**

557

558 • **SELECT * FROM Patient_Transaction;**



The screenshot shows a database client interface. At the top, a SQL query is entered: **SELECT * FROM Patient_Transaction;**. Below the query, there is a toolbar with icons for 'Result Grid', 'Filter Rows', and 'Edit'. The 'Result Grid' is active, displaying a table with the following data:

Patient_ID	Patient_Name	Age	Disease
1	Arun Kumar	25	Fever
NULL	NULL	NULL	NULL

Result:

The commands SQL COMMIT, ROLLBACK and SAVEPOINT are implemented and result is verified.